

State Machine Specification

4-bit Shared State Machine (One Transition – Two directions)

Figure 1 shows the reversible state machine model for a *single* link. Alice (yellow) and Bob (blue) are two entities in a ‘causal conversation’. The stop and wait (SAW) – ping-pong protocol guarantees properties that we can turn into useful tools for building *ultra-reliable* distributed systems.

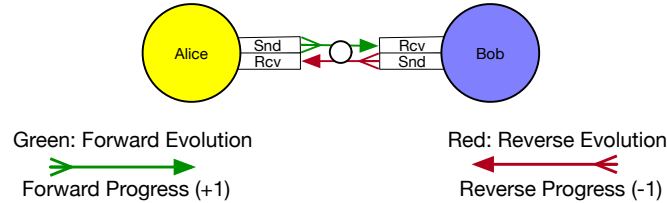


Figure 1: Single Link(4-bit Dynamic Spekkens Liveness)

The Shared State Machine (Fig. 2) has 8 primary states and 16 transitions.

Each transition has three sub-states (initial conditions, action, and final conditions), represented by Snd, →, and Rcv respectively. The green channel drives the state machine clockwise, and the red channel drives the state machine anti-clockwise. Closing the protocol in pairs guarantees that information is not lost.

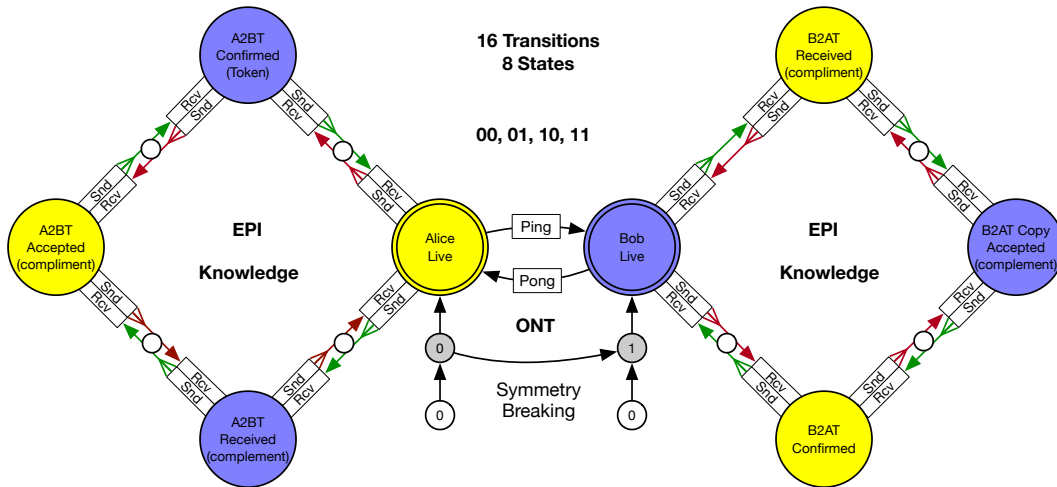


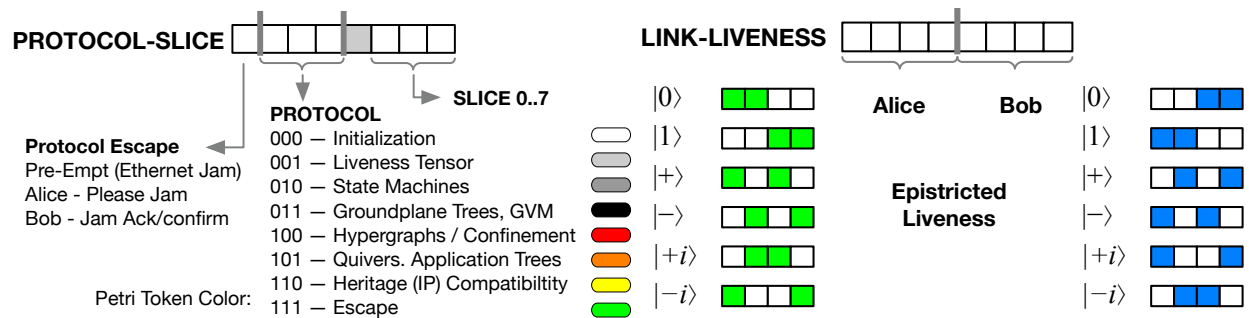
Figure 2: Reversible 4-State Machine Model with Ping Pong

The State Machines provide:

- Symmetry breaking Initialization / Authentication (race condition resolver).
- A Liveness Tensor for consensus link health status one or two hops out (before tree stacking).
- A state Machine Engine (this document)
- Treebuilding and Healing Protocol.
- Hypergraph (Hypercell clusters with self-cell and cells connected to 8 ports).
- Quivers, Causal data structures and ARROWs.
- Heritage (L3 Internet Protocol)Compatibility

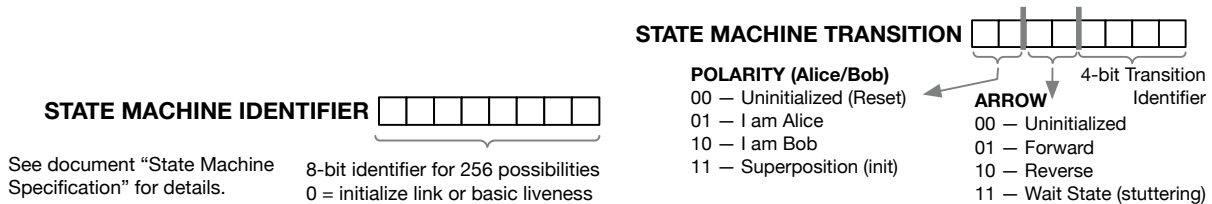
Protocol [010]

This specification describes the [010] (State Machines) PROTOCOL. From the SLICE ENGINE Document:



ARROW The current direction of the State machine. The forward/backward progress of the state machine (ancilla provided by error processing code).

POLARITY Defines whether we are Alice or Bob (from protocol [000] (Autoconfiguration/Authentication)).



STATE MACHINE IDENTIFIER Which of the 256 possible state machines are in effect for this message (should be consistent over multiple slices and responses).

STATE MACHINE TRANSITION Defines which of the (up to 16) transitions the shared state machine is currently in – ensures both sides of the link are synchronized.

There are 256 Possible State Machines. The first of these are defined in the following pages (1-state, 2 state, ... n-state), etc. A consistent set of protocol fields and roundtrips are specified in Petri-Net models for each of the 256 Possible State Machines.

Protocol Phases and States

There are three main dimensions to the design:

- The number of slices processed by the FPGA in the incoming link before it emits a response on one or more of the outgoing links (including link zero – the path up to the Host processor on the other side of the PCIe bus).
- The number of round trips expected in the protocol. The simplest principle is conservation of information: you cannot read (observe) a variable more than once. This is not just physics, it must be built into our implementation. Every write has to be interlocked (paired) with every read (transfer not copy). We can't cheat and "observe it", such as by conventional styles of debugging, writing to stdout or viewing in a browser. That would not be conserved information.
- The number of bits in the Spekkens Metadata, the four bit protocol (2 entangled toy qubits), describes 16 possible states, only 6 of which are correct states. The remaining 10 inconsistent states are used to test for errors.

Petri Net Models

Finite State Machines (FSM's) are sequential – only one state can be active at a time. This document uses Petri-Nets for the details, which allow multiple parts of the model to be run concurrently with different tokens and race conditions. It is the race conditions that represent the 'ONT' (inaccessible) information. See the document 'Dual Petri Epistricted Liveness Protocol' for details.

Tokens

Tokens are first class citizens. They are 'mechanistic' things, whose creation and deletion are carefully controlled (for robustness in the face of failures). We define the following kinds of tokens:

Physical Tokens things that exist and we can uniquely relate to in the outside world. They are unique, countable and durable against destruction. They are 'ownable'.

Logical Tokens have a cryptographic identity, created using a Physically Unclonable Function¹ They are unique and 'ownable', and made durable against all known hazards. Copies (backups, remote copies, cached copies) are precisely counted and distinguished.

Virtual Tokens are locally unique. They are carefully generated and retired within the link protocols.

Token conservation conservation as giving a guarantee that these distributed representations will act together as a single token.

1. No two created tokens are equal; and
2. No token may be used after it has been destroyed.
3. If tokens are to have mutable state, that state must be maintained consistently across representations (for some definition of consistency).

We reserve the right to copy representations of semantic tokens (for robustness in the face of failures) and presumably transmit those copies across various connections. We like to think of conservation as giving a guarantee that these distributed representations will act together as a single token. If a token is a piece of information being transferred, it is created upon message send and destroyed upon confirmed receipt. What would we then mean by conservation?

¹A physical unclonable function (PUF) is a device that exploits inherent randomness introduced during manufacturing to give a physical entity a unique 'fingerprint' or trust anchor. See: [Physical unclonable functions](#) and [Hybrid Boolean Networks](#).

1-State Transfer State Machine [001] Liveness

The simplest and most common state machine – ‘LIVENESS’. Opening TRANSITION $zero$ $[001]$ Closing TRANSITION $\overline{zero}$. $[101]$. Yellow circles identify Alice places; Blue circles identify Bob places. When there is no activity on the link, we continuously maintain liveness with this mechanism.

Liveness Protocol – Timeless

1. *zero*. Opening symmetric liveness query.
2. $\overline{\text{zero}}$. Closing symmetric liveness query.

2-State Transfer State Machine [010]

The 2-State transfer (2ST) protocol is a simple send / acknowledge. It transfers a single token (colored here in red/French) to the other side of the link. The token could be a simple data payload. Not a transactional payload. The Initialization/Symmetry breaking Petri net (included in the first example above) is assumed in this and all Petri Net's below. Reverse protocol is trivial – simple rejection and ‘undo’ of the operation.

1. *un* Alice speaks to Bob, confirms symmetry is broken. Understood consistently on both sides of the link.
2. *over*, when Bob Speaks to Alice. This enables error-checking for the symmetry-breaking protocol.

4-State Transfer State Machine [011]

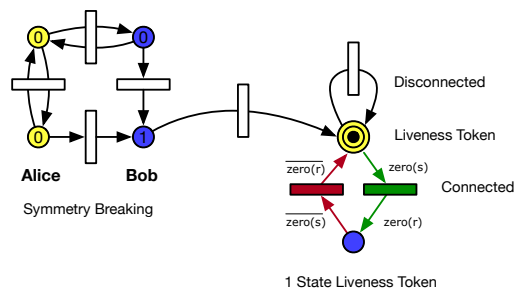
The 4-State Petri net protocol requires 4 messages to complete. The protocol is specified in german number labels to signify that a blue (german) token is being transferred across the link.

Forward DIRECTION [01]

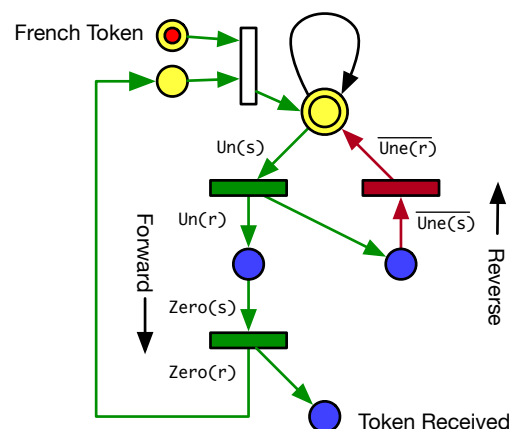
1. *eins* Alice has a token she wishes to send to Bob, and puts a different color token on the wire.
2. *zwei* Bob receives and sends; Bob-saw-token.
3. *drei*. Bob responds to Alice, now Alice knows that Bob Knows, that the token is there.
4. *vier* Transaction complete. Alice now knows that Bob knows, that Alice knows he has the token.

Reverse DIRECTION [10]

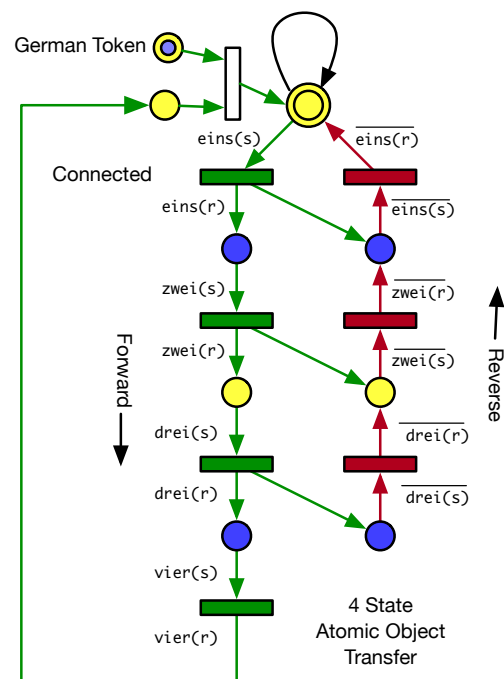
1. $\overline{eins}$ when Alice speaks to Bob, $\overline{une}$, when Bob Speaks to Alice. This enables error-checking for the symmetry-breaking protocol.
2. $\overline{zwei}$ Undoing operation *un*
3. $\overline{drei}$ Undoing operation *deux*
4. $\overline{vier}$ This operation is not needed. If the protocol reaches the *vier* state, then the final operation is *eins*, which completes the transaction and flushes the token.



1-State Petri net Model (Liveness)



2-State Transfer Petri net Model



4-State Petri net Model (Liveness)

8-State Shared State Machine. PROTOCOL [010] IDENTIFIER [100]

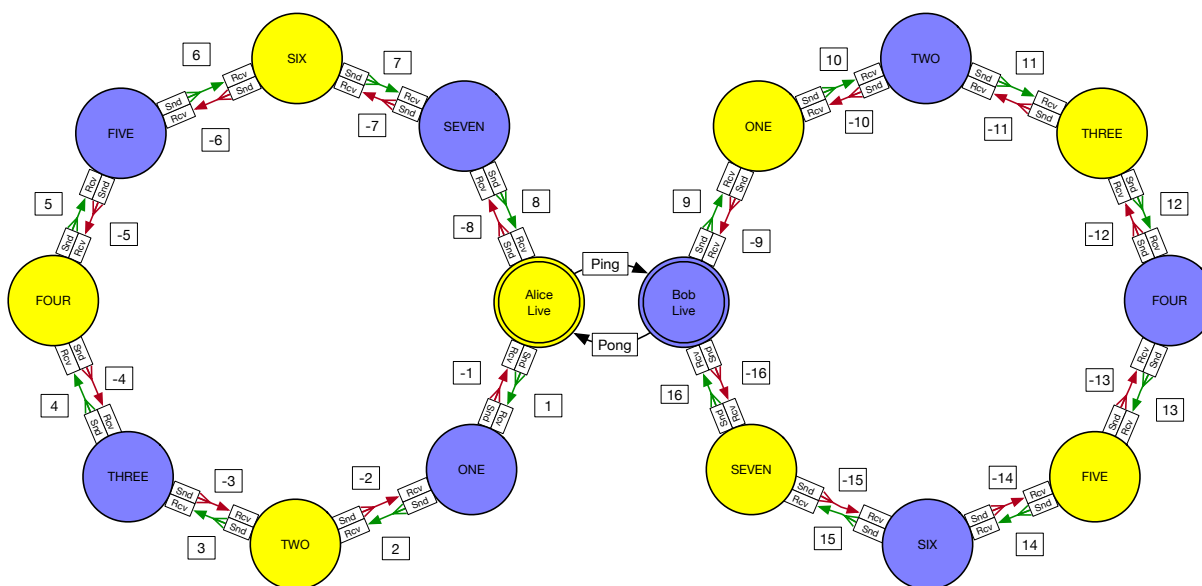


Figure 3: 8 State Shared State Machine

There is only one path back through the state machine if the token transfer is rejected. This is not a factorial (exponential) expansion of state space, it is linear (proportional to the number of states in the forward path). At most, it doubles the complexity of the basic mechanism. This operation is not reversible.

This protocol (and one of several other protocols characterized by the color of the token) may be used for purposes chosen by the transaction system. For example, in one language (say german), the protocol 'lends' the token to the other side of the link, while maintaining knowledge on this side of the link that we still 'own' the token, and expect to get it back when the other side of the link has finished using it.

8-State State Machine [100]

Forward DIRECTION [01]

1. *eins* Begin 1PC.
2. *zwei* Completion of 1PC.
3. *drei* Begin 2PC
4. *vier* Completion of 2PC
5. *funf* Begin 3PC – token lending (borrowing) initiated across link.
6. *sechs* Completion of 3PC, lending (borrowing) completed across link.
7. *sieben* Begin 4PC – token ownership transfer (initiated across link).
8. *acht* Completion of 4PC. Token successfully transferred across link.

Reverse DIRECTION [10]

1. $\overline{eins}$ when Alice speaks to Bob, $\overline{une}$, when Bob Speaks to Alice
2. $\overline{zwei}$ Each of these steps is simply a place to ‘undo’ the operations, and ensure that there is no residual information that can pollute the system.
3. $\overline{drei}$
4. $\overline{vier}$
5. $\overline{funf}$
6. $\overline{sechs}$
7. $\overline{sieben}$
8. $\overline{acht}$

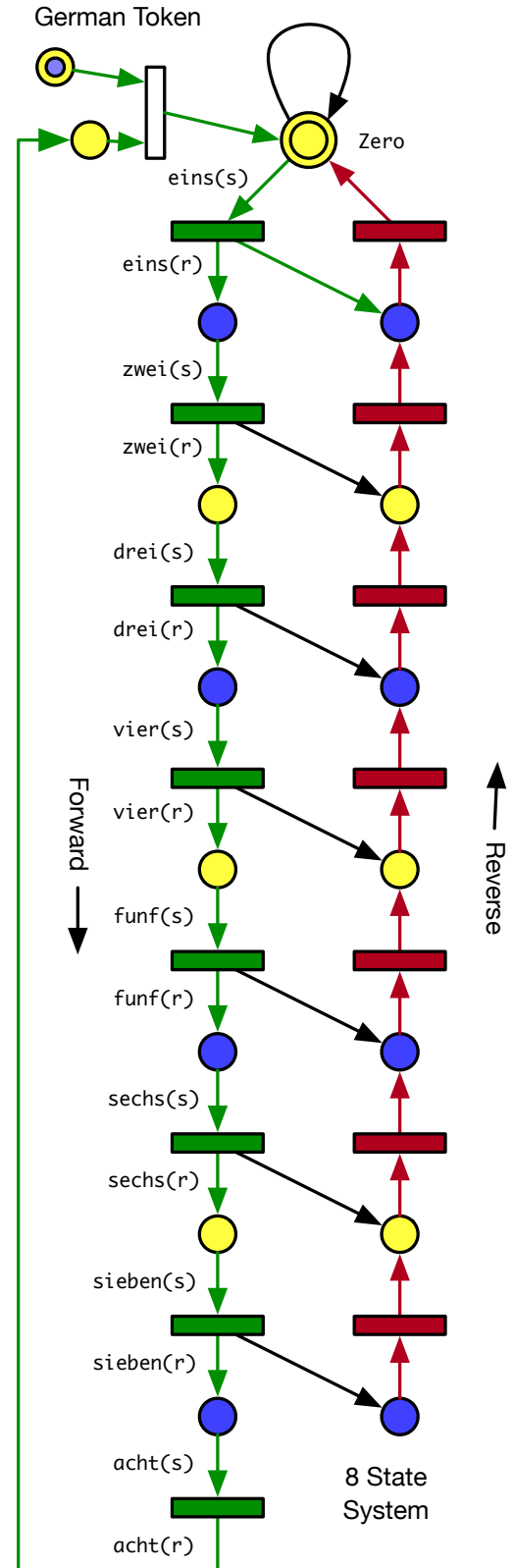
Uninitialized DIRECTION [00]

This condition indicates an error. The DIRECTION State was not initialized before being used. This should never occur.

Wait State – Stuttering DIRECTION [11]

This condition indicates the a single wait state. Typically the protocol will use this as a single step wait state before going from forward to reverse, or from reverse to forward.

Stuttering is an artificial failure mode. It should occur for only one cycle before being automatically shifted back to forward or reverse.



8-State Transfer Protocol

Petri Protocol Phases

There are three phases in the bit commitment protocol: Preparation, Transformation and Measurement. These correspond to the three transition elements on the wire from each place in the figure above.

While this condition of irreversibility (once the commitment phase is ended, there is no way for Alice to alter her choice). However, in a cluster of cells with Alice (*A*) at the center, and Cells *B, C, D, E, F, G, H* and *I* connected to the 8 ports on Alice, there is a question of *when* cells *B* through *I* have also decided. In principle, any one of those 8 cells could delay the entire cluster from coming to agreement.

We solve this using [Flexible Paxos](#), where the set of cells in the first phase can be *somewhat* disjoint from the set of cells in the second phase, and we don't care, for example, if one, or two, or three of the cells take a long time to make a decision. We simply take the remaining 3, 5 or 7 cells as the decision, and turn the straggler into 'follower' from the Paxos perspective.

The Link State Petri net has 6 places (3 for Alice and 3 for Bob). This 'high-level' Link State is available for control and communication with the next layer of software above. The four primary Link states are:

- [00] Stop
- [01] Run
- [10] Initialize.
- [11] Liveness (Superposition).

The above binary states are derived from the eState ('bits on the wire' and 'bits off the wire') as follows: we label bits as bit 1 (Alice) and bit 2 (Bob). The informational question being asked is "I'm OK, are you OK?". Alice does this by setting her bit 1 to true, and bit 2 to 0. The bits are then transmitted on the wire, and the result is stored in the eState on Bob's side. The Petri action sets Bob's Bit True, and stores the result in Bob's Link State. The result is that, when the link is live, Both Alice's, and Bob's bits are TRUE (11) which indicates a state of liveness (or superposition). After populating Bob's Link State, the Petri action now set's Alice's presence bit to TRUE, and stores this new 2 bit state in Bob's eState, prior to sending the bits on the wire.

The reciprocal action is now taken by Alice on the packet received from Bob. This should create a situation where the Link State for both Bob and Alice are always 11, and the eState for Bob should be 1001, and the eState for Alice is also 1001.

This particular situation can more easily be understood from the perspective of low-level Petri Tokens. There are two Petri tokens involved in each transfer.

Cluster Level Transactions

Fig. 4 Shows a combined state machine view from one node, looking out on the state of 6 links. The inner tokens (black and white) are updated only if the black and white tokens are changed on the other side of the link. i.e. The links are self-contained, and silent; until some change occurs on the other side of the link.

The meaning of the black and white tokens in a quiescent system is simply the link status. We 'could' color them grey to distinguish them from knowing that they are in a state of ping or pong. This then paves the way for using Colored tokens as the actual means of information transfer.

Petri net description. The circles are called places. They describe the states of the system. The rectangles are called transitions. They describe the actions. The arrows are called arcs. The arc expressions describe how the state of the CP-net changes when the transitions occur.

Coloured Petri Nets ² have got their name because they allow the use of tokens that carry data values and can hence be distinguished from each other – in contrast to the tokens of low-level Petri nets, which by convention are drawn as black, 'uncolored' dots.

Fig. 4 shows an example RNT mechanism. Each link provides individual coherence of tokens across each link. In a cluster, the RNT algorithm can coordinate the event order on multiple links to meet the constraints (invariants) of the application (e.g. Reversible Nested Sub-transaction, or consensus operation).

²Jensen & Kristensen [Colored Petri Nets: A Graphical Language For Formal Modeling and Validation of Concurrent Systems](#).

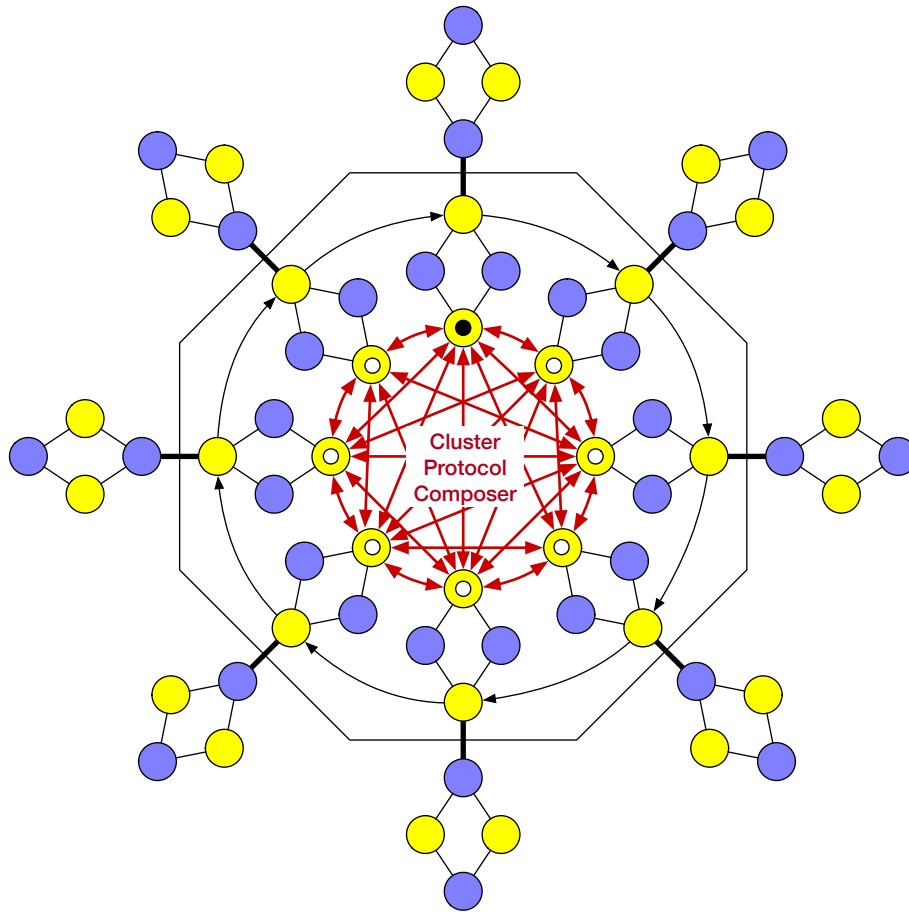


Figure 4: Cluster Protocol Controller Combining 8 links and a single coordination thread.

4-bit protocol

Alice needs 2 bits to specify her relationship to the link. The exact description isn't as important, but a practical description is that [00] means the link is disabled/disconnected [01] is one kind of state (say ping or tick), [10] is the other kind of state (say pong, or tock), and [11] represents 'unknown', or 'in superposition'. If Bob also has 2 bits to specify his relationship to the link, the description is the same as above.

The fundamental test is when you mix (add or combine) both Alice's and Bob's bits (so you get 4 bits total, but you don't know which is which), then you will get 'mutual information'. This is ONT information. All you need to know is that the link is in superposition, not whether it is tick or tock in that instant.

The most that can be achieved with this 4 bit system is 'liveness'. There is no actual information transfer, but there is 'common knowledge': the answer to a very simple yes/no question: 'are you there'. In addition to both parties knowing that the transfer is complete, they both know that they both know that it is complete. That fact is important for finalizing transfers.

Token Borrowing (lending), and Token Ownership (transfer, not copy)

8-bit protocol

By adding more bits to the protocol we can extend it in the same way to add other forms of common knowledge. The 4 bit protocol is simply about presence management (liveness – are you there? – yes I am, are you there?). While the 4-bit protocol is simply the concatenation of two 'qubit'- like things, the 8-bit protocol extends this to manage Conserved Quantities (CQ) in the link, using:

Token lending (the complementary state of the token is held on the near side of the link when the token is 'lent' to the other side), but the ownership is 'locked' on the other side until the remote entity has finished using it. It takes a 2-phase (4-event) protocol to 'lend' a token. This makes sure a representation of the token exists on both sides of the link, and uniquely identifies which one is the owner.

Token ownership transfer The token is no longer borrowed, its ownership is transferred completely and irreversibly to the other side. It takes an additional 2-phase protocol (total 8 events for closure) to move the ownership from one agent to another.

Consistency

Most operations on data are correct (create consistent results) only when the ordering of the applied operations is a certain way (non-commutative). Rather than accept some arbitrarily arrival order at some random node in the network, we can specify (using a sub-transaction schedule) what the 'acceptable' orderings are. Instead of there being one correct ordering (out of dozens of possible orders), can there be 'multiple' possible orderings that are correct (with many of them still incorrect)? If the answer is that there can be more than ordering that provides a 'consistent' alternatives that the system can settle into. This particular model is referred to as '[Multiway Systems](#)³'. See The document 'Liveness Tensor' for details.

The reason that the encoding is somewhat arbitrary (not numbers, binary or decimal) is because the protocols all work by graph [rewriting](#)⁴. There are 3 levels in the power of the computational models: (1) Cellular Automata is the weakest. 2. Graph rewriting is more powerful than CA. 3. Multiway graph rewriting is more powerful than simple graph rewriting. In a Graph, a directed ARC from one element to another is a "possibility" in the computation.

Multiple Layers of Reversibility

Fig. 5 Shows the link in the physical connection between two computers (FPGA Event Logic Between Alice and Bob), and two additional reversibility layers above. Each with a 'link-equivalent' hidden liveness mechanism. In a single machine with multiple cores connected on a common bus, this is achieved through conventional issue of instructions which cause cache coherency operations to be emitted to synchronize the caches and prevent stale data.

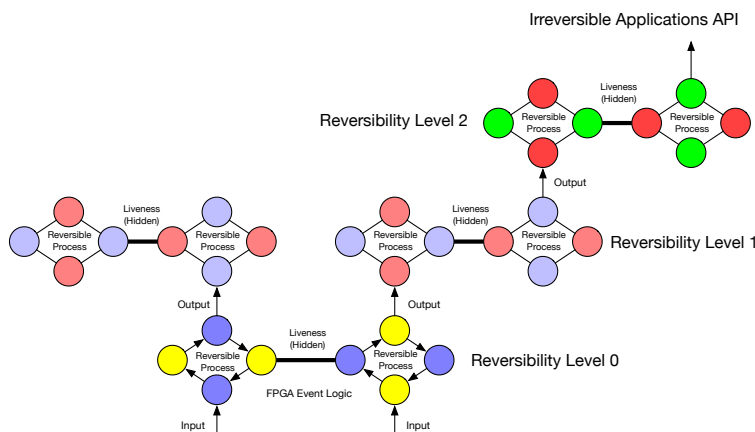


Figure 5: Multiple Layers of Reversibility

³A in a multiway graph an edge can have more than 2 points on its arc. Instead of a 2-tuple (node, edge) a directed arc has an intermediate node. See rigorous definition of hypergraph on [A hypergraph is a graph in which generalized edges \(called hyperedges\) may connect more than two nodes.](#)

⁴Rewriting systems then do not provide an algorithm for changing one term to another, but a set of possible rule applications. When combined with an appropriate algorithm, however, rewrite systems can be viewed as computer programs, and several theorem provers and declarative programming languages are based on term rewriting.

Include this

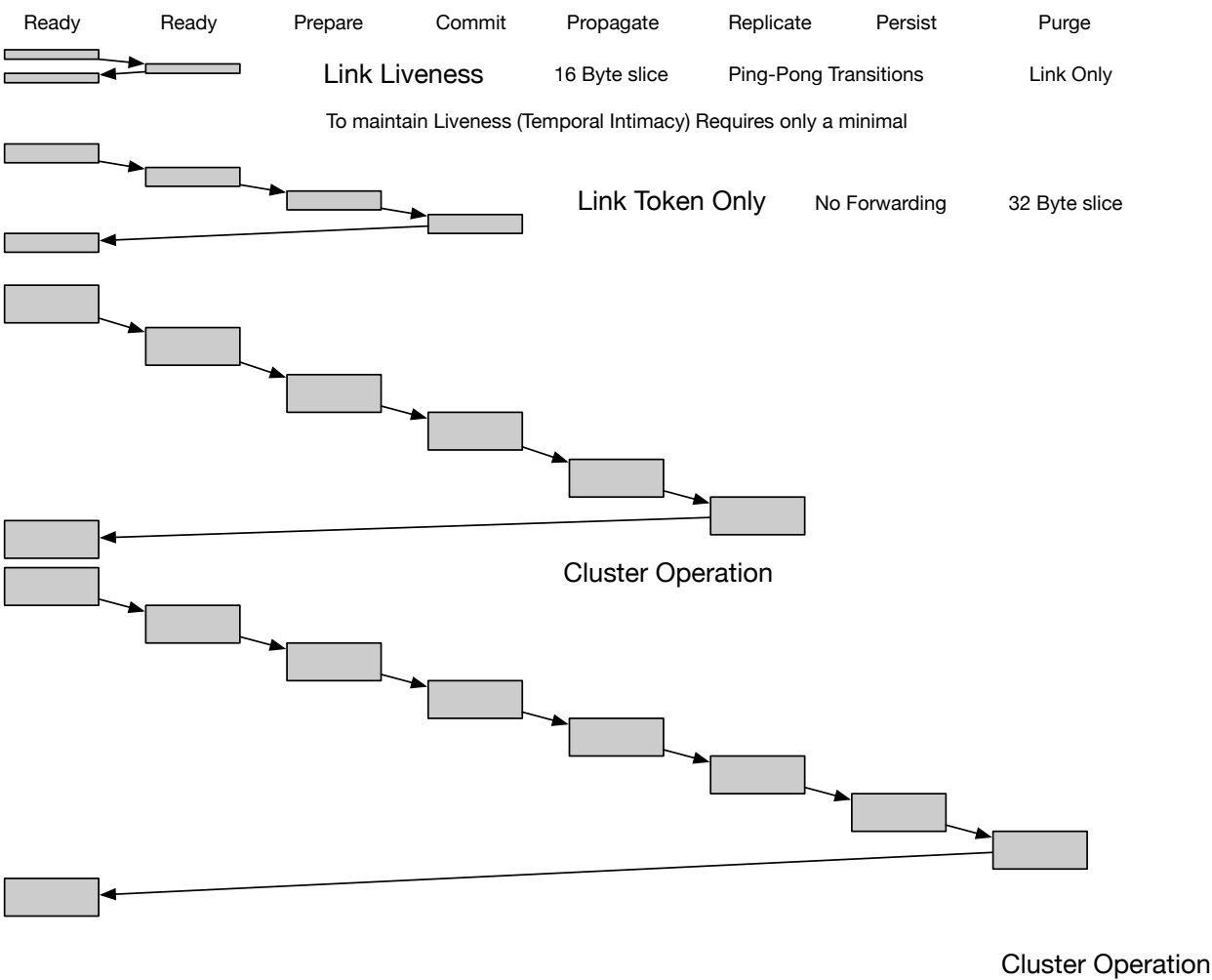


Figure 6: Cluster Operation – to be included